

Sound Data-Flow Verification for Safe Serverless Workflow Orchestration: Provenance, Tpestate, Concurrency, and Compensation

Anonymous Author(s)

Anonymous Institution
anonymous@example.org

Abstract. Serverless workflow orchestration platforms such as AWS Step Functions coordinate fleets of functions and managed services, but their workflows are written as untyped JSON state machines (the Amazon States Language, ASL) whose correctness is checked only at run time. The validators developers have today—`statelint`, `asl-validator`, and AWS’s own `ValidateStateMachineDefinition`—check JSONPath *syntax*, not whether the field a path names can ever have been *produced*, whether tasks run in a valid business order, whether a retried operation is safe to repeat, whether concurrent branches contend on a resource, or whether a retry can complete within its timeout. We present STEP CHECK, a static verifier whose centrepiece is, to our knowledge, the first *sound* data-flow analysis of the ASL I/O-processing pipeline: a flow-sensitive abstract interpretation over a may-present lattice of document shapes that reports a missing-field reference only when it is guaranteed to fail at run time (no false positives), recovering the classic “**amount** vs. **total**” contract bug with zero annotations. Around it STEP CHECK adds six further checks across structural, tpestate, retry-safety, *concurrency-interference*, *retry/timeout-budget*, and Saga-compensation analyses—two of them, concurrency and temporal budget, addressing classes no ASL tool reasons about. Because raw ASL omits task semantics, STEP CHECK pairs the sound native analyses with a lightweight annotation layer whose defaults are *inferred* from naming and resource conventions, so existing workflows are analysed at zero up-front cost and refined incrementally. Implemented in Rust as a modular frontend/pass pipeline (3.7k LOC, eight passes), STEP CHECK is evaluated on **193** real Step Functions workflows: the data-flow and concurrency analyses fire zero false positives in the wild (corroborating soundness), the temporal analysis surfaces 33 unbounded-callback defects, and a **616**-mutant study detects every injected defect with **100%** recall—where the `statelint` validator catches none of the unsafe retries, missing compensations, concurrency, or temporal defects. Inference recovers persistence at 86% but idempotency at only 62% against an agreement-checked gold set, so semantic findings are surfaced as warnings; verification costs $\approx 39\mu\text{s}$ per workflow; a verified workflow deploys and runs unmodified on AWS Step Functions with zero runtime overhead; and the architecture generalizes to a second workflow language (CNCF Serverless Workflow) with no change to the analyses.

Keywords: Serverless computing · Workflow orchestration · AWS Step Functions · Static verification · Data-flow analysis · Abstract interpretation · Typestate · Saga transactions.

1 Introduction

Cloud-native applications increasingly decompose business logic into independently deployable functions that are coordinated by managed *workflow orchestration* services. Among these, AWS Step Functions has become one of the most widely adopted platforms in industry [20,13]. A Step Functions workflow is a state machine written in the *Amazon States Language* (ASL), a JSON configuration format that describes task invocations, branching, parallelism, retries, and error handling, and that executes on top of AWS Lambda and dozens of service integrations [2].

While orchestration platforms simplify *running* distributed workflows, they provide little support for *reasoning about workflow correctness before deployment*. An ASL state machine is an untyped artifact, and the tooling developers have—`statelint` [3], `asl-validator`, and AWS’s own `ValidateStateMachineDefinition`—validates the JSON schema, the existence of transitions, and the *syntax* of JSONPath expressions (“does this look like a path?”). None of them reasons about whether the field a path *names* can ever have been produced by an upstream state, whether tasks are invoked in a valid business order, whether an automatically retried task is safe to repeat, whether concurrent branches contend on the same resource, or whether a long-running transaction can roll back the effects it has committed. As a consequence, six recurring classes of defect surface only at run time (detailed in Section 2.2): *contract / data-flow mismatches* (a task reads a field no predecessor produces—the running “`amount` vs. `total`” bug); *protocol/typestate violations* (a task runs out of business order—shipping before payment); *unsafe retries* (a retry re-invokes a non-idempotent operation, duplicating its effect [14]); *concurrency interference* (two `Parallel` branches, or concurrent `Map` iterations, write the same external resource); *temporal-budget faults* (a `waitForTaskToken` callback that can hang indefinitely, or retries that cannot complete within a timeout); and *missing compensation* (a persistent operation cannot be rolled back when a later step fails—an incomplete Saga [11]).

A central obstacle to checking these properties is that ASL *omits the very information the checks require*. ASL has no notion of a task’s input or output type, no business-state labels, no idempotency annotation, and no concept of “compensation” beyond a generic `Catch`. Existing language-level approaches sidestep this by introducing a new programming model with a proprietary runtime [6,21,18]; but the millions of workflows already written in ASL cannot benefit, and re-platforming is rarely an option.

Approach. We present STEP CHECK, a static verifier that runs *before deployment* while remaining fully compatible with AWS Step Functions. Its intellectual centrepiece is a *sound, flow-sensitive data-flow analysis* of the ASL I/O-processing

pipeline (`InputPath` $\rightarrow$ `Parameters` $\rightarrow$ `ResultSelector` $\rightarrow$ `ResultPath` $\rightarrow$ `OutputPath`). Cast as an abstract interpretation over a *may-present* lattice of document shapes, it tracks which fields a state’s document can hold and reports a reference to a definitely-absent field—with *no false positives by construction*. This moves the contract check from JSONPath *syntax* (what every existing validator does) to field *provenance*, catching the “`amount` vs. `total`” bug with zero annotations. STEP CHECK is organised around two further ideas. First, a *two-layer* model: native checks (structural well-formedness, data-flow provenance, concurrency interference, temporal budget) are sound and need no input, while semantic checks (typestate, retry safety, compensation, typed contracts) consume a lightweight *annotation* layer. Second, because most workflows carry no annotations, the missing semantics are *inferred* from naming and resource conventions, giving a zero-effort default that developers can override. Verified workflows are compiled back to executable ASL, so STEP CHECK adds no runtime component and no lock-in; a typed builder frontend additionally lets greenfield workflows be authored with full static guarantees.

Contributions.

- **A sound data-flow / field-provenance analysis for the Amazon States Language**, formalized as an abstract interpretation over a may-present lattice with a no-false-positive guarantee—to our knowledge the first static analysis of ASL data flow beyond JSONPath syntax. It has a native tier (no annotations) and a typed tier (declared input schema) and subsumes the syntactic binding checks (Sections 3.3, 4).
- **Two further analyses for classes no Asl tool addresses**: *concurrency interference* (`Parallel/Map` branches that write the same external resource) and a *retry/timeout-budget* (temporal) analysis (unbounded callbacks, retries that cannot complete within a timeout, heartbeat/timeout mis-configuration).
- A two-layer design that verifies *existing* ASL by pairing sound native checks with an annotation layer whose defaults are inferred from naming/resource conventions, together with the structural, typestate, retry-safety, and compensation checks—eight passes in all (Sections 3–4).
- STEP CHECK, an open, modular Rust implementation (3.7k LOC, eight passes) with pluggable frontends (raw ASL, a typed DSL, the CNCF Serverless Workflow DSL), a diagnostics engine, an ASL emitter, and a mutation-based error-injection tool (Section 5).
- An evaluation on **193 real** workflows (plus 66 CNCF): the data-flow analysis fires *zero* false positives in the wild (soundness), the new analyses surface genuine in-the-wild defects, a **616**-mutant study detects every injected defect (100% recall) where `statelint` catches none of the retry, compensation, concurrency, or temporal classes, plus inference accuracy and warning precision against an agreement-checked gold set, a typed-tier data-flow recall study, sub-millisecond cost, and a real AWS round-trip (Section 6).

Research questions. **RQ1:** Can a static verifier detect these defect classes in serverless workflows before deployment—and how does it compare to the validator

developers already use? **RQ2:** Is the data-flow analysis sound in practice, and can the semantics ASL omits be recovered cheaply enough to analyse existing workflows? **RQ3:** What is the cost of verification and what runtime overhead does it impose? We answer these in Section 6.

2 Background and Motivation

2.1 Step Functions and the Amazon States Language

A Step Functions workflow is a state machine: a **StartAt** entry and a map of named *states*. A **Task** state invokes a Lambda function or a service integration; **Choice** branches; **Parallel** and **Map** fan out; **Wait**, **Pass**, **Succeed**, and **Fail** complete the set. Data flows through a JSON document that each state reshapes with JSONPath fields (**Parameters**, **InputPath**, **ResultPath**, **OutputPath**), where a key suffixed `.$` denotes a path reference. States carry retry (**Retry**) and error-handling (**Catch**) policies. Listing 1.1 shows a representative task drawn verbatim from our corpus.

Listing 1.1. A real ASL task state (excerpt, `ship-order`).

```
"Get Customer Status": {
  "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
  "Parameters": { "FunctionName": "${LambdaGetCustomerStatus}",
                  "Payload.$": "$.detail.customer" },
  "Retry": [ { "ErrorEquals": ["Lambda.ServiceException"], "MaxAttempts": 6 } ],
  "Next": "Do Fraud Check"
}
```

The platform validates such a definition’s JSON schema and that every transition target exists, then executes it. It does *not* reason about whether `$.detail.customer` is present, whether retrying this task is safe, or how the workflow recovers if a later step fails.

2.2 Six classes of latent defect

Our running example is the canonical order-processing workflow `CreateOrder` $\rightarrow$ `ReserveInventory` $\rightarrow$ `ChargeCard` $\rightarrow$ `ShipOrder`, which exhibits six defect classes when edited carelessly. (1) *Contract / data-flow*: `ReserveInventory` produces `{orderId, amount}` but `ChargeCard` reads `$.total`—a run-time failure on a missing field. Crucially the path `$.total` is *syntactically* a perfectly good JSONPath, so every schema/syntax validator accepts it; only reasoning about *provenance*—which fields the document can hold at that point—exposes it. (2) *Typestate*: reordering to `ShipOrder` $\rightarrow$ `ChargeCard` ships before payment (valid JSON, invalid protocol). (3) *Retry*: non-idempotent `ChargeCard` with a broad retry (`States.ALL`) may charge twice. (4) *Concurrency*: a **Parallel** that runs `ChargeCard` and a fraud-screen branch which both write the same DynamoDB row, or a **Map** whose concurrent iterations all `PutItem` to one fixed key, races

on a shared resource. (5) *Temporal*: a `waitForTaskToken` approval step with no `TimeoutSeconds/HeartbeatSeconds` blocks the execution for up to a year if the token is lost. (6) *Compensation*: `ReserveInventory` commits a reservation that leaks if `ChargeCard` fails with no release path. None is caught by the platform; all six are recognised, under-tooled failure modes in cloud-native systems [14,13]. We claim nothing about their prevalence—curated examples are mostly correct (Section 6.2)—only that they escape static checking entirely today; Section 6 measures detection on both unmodified and injected defects.

2.3 The information gap

The obstacle to checking these properties statically is not the analysis but the *input*. Raw ASL encodes control flow and JSONPath data plumbing, but it does not record: the *type* of data a task consumes or produces; the *business state* a task represents; whether a task is *idempotent*; or whether a task is *persistent* and which action *compensates* it. A verifier for existing workflows must therefore either (a) restrict itself to properties decidable from ASL alone, or (b) recover the missing semantics. STEP CHECK does both: it runs sound native checks that need nothing beyond ASL, and it recovers the remaining semantics through a lightweight annotation layer whose defaults are inferred from the strong naming and resource conventions that pervade real workflows (Section 3).

3 Approach

3.1 A two-layer analysis model

STEP CHECK separates what can be checked from ASL alone from what needs recovered semantics. *Native* analyses (Layer A) are sound and require no annotations: structural well-formedness; the *data-flow provenance* analysis of Section 3.3 (which subsumes the ASL-semantic binding checks—a `waitForTaskToken` task that never passes its token, `$$`.Map misuse, invalid JSONPath payloads—and adds field-availability reasoning); the *concurrency-interference* check (`Parallel/Map` branches that write a shared resource); and the *retry/timeout-budget* check (unbounded callbacks, retries that overrun a timeout, heartbeat/timeout misconfiguration). *Semantic* analyses (Layer B)—typestate, retry safety, compensation, and typed contract compatibility—consume an *annotation* layer that records the task semantics ASL omits. (Concurrency and retry safety also *read* the annotation layer to learn which tasks write or are non-idempotent, so they sharpen as annotations are added, but their structural triggers fire natively.) The annotation layer has two sources, in priority order: explicit annotations (authored by hand or produced by the typed DSL frontend), and *inference* that supplies defaults from naming and resource conventions. Findings derived from inferred facts are reported at *warning* severity, while findings derived from declared facts are *errors*; this single rule keeps confidence visible throughout the tool’s output. We are deliberate about what “verification” means at each layer: the native checks

are *sound* and need no input, whereas the semantic checks are only as strong as the semantics they are given—with declared annotations they are precise errors, and in the zero-annotation default they degrade gracefully to best-effort, inference-driven *warnings* (typestate, having no labels to act on, simply stays silent). STEP-CHECK is thus a sound linter that becomes a precise verifier as annotations are added.

3.2 Workflow model

We model a workflow as a graph of states with a designated start. Beyond its control-flow successors, a *task* carries four pieces of semantic information. **Contract:** a task $T : A \rightarrow B$ consumes/produces record types (field sets); adjacent $T_1 : A \rightarrow B$, $T_2 : C \rightarrow D$ *compose* only if B provides every field C requires (natively, without declared types, the layer instead checks that JSON-Path references resolve to data the workflow binds). **Typestate:** T advances a *business state* $s_{\text{in}} \rightarrow s_{\text{out}}$; a workflow is well-ordered when each task consumes the state its predecessor produced, and an optional *protocol* (e.g. *created* $\rightarrow$ *reserved* $\rightarrow$ *paid* $\rightarrow$ *shipped*) lists permitted transitions. **Idempotency:** repeating an idempotent task causes no extra external effect, so it may be retried freely; a non-idempotent one may not be retried on broad error classes. **Persistence/compensation:** a *persistent* task commits a durable resource (reservation, charge, created record) that a later failure should undo, so it needs a *compensation*—an explicit handler or a **Catch** to a compensating state. This vocabulary is deliberately lightweight—no separate formal calculus—yet exactly what the semantic checks of Section 4 consume.

3.3 Sound data-flow provenance by abstract interpretation

The contract defect of Section 2.2 is a *data-flow* fact: `$.total` fails not because it is malformed—it is a valid path—but because no execution reaching **ChargeCard** produces a `total` field. Deciding this requires modelling how ASL reshapes its state document, which it does through a fixed five-stage pipeline per state: **InputPath** selects a sub-document; **Parameters/ItemSelector** *construct* the task input (each `"k.$":"$.p"` entry *reads* `$.p`); the task result is combined through **ResultSelector** and merged into the *raw* input at **ResultPath**; and **OutputPath** selects the state output. We give this an abstract interpretation.

Domain. A document shape over-approximates the set of field paths that *may* be present:

$$\sigma ::= \top \mid \{k_1:\sigma_1, \dots, k_n:\sigma_n\}$$

where $\top$ is opaque (any field may be present—e.g. an arbitrary Lambda result or the execution input) and a record lists *exactly* the keys that may occur (a key absent from the record is absent on every represented document; scalars/arrays are the empty record / $\top$). The order is $\sigma \sqsubseteq \top$ and pointwise on records; the join $\sqcup$ is the *union* of keys ($\top$ absorbs), so a key present on any incoming path survives. A path `$.p1...pk` is *definitely absent* in σ iff descending the records hits a missing key without first reaching $\top$.

Transfer functions. Each stage is a monotone function on shapes: `InputPath/OutputPath` re-root by descent (a missing or non-trivial selection yields $\top$); `Parameters/ItemSelector/ResultSelector` build a record whose keys are the constructor’s keys (a `.$` leaf reads an opaque value, hence $\top$); the opaque task result is $\top$ unless a `ResultSelector` pins its keys; `ResultPath` places the result into the raw input, preserving siblings (`$` replaces, `null` discards). A standard forward fixpoint over the control-flow graph—joining at merge points and along `Catch` edges (which carry an opaque error object)—computes an entry shape for every state. We then check exactly the references the ASL runtime *evaluates and aborts on when absent*: the ‘`$`’ constructors of `Parameters/ItemSelector` and a Map’s `ItemsPath`. We deliberately do *not* check `Choice` operands: a `Choice` comparison (`IsPresent`, `IsNull`, value comparators) is a *test* that returns false on an absent field rather than failing the execution—flagging it would be unsound.

Soundness. The analysis reports `SC1101` for a reference only when its path is definitely absent in the state’s entry shape. Because every transfer function over-approximates the set of may-present paths and $\top$ absorbs all uncertainty, this yields:

If `STEPCHECK` reports `SC1101` for path p at state s , then in every concrete execution that reaches s , p is absent from the state document—so the reference fails at run time.

Equivalently, the analysis has *no false positives*: it never flags a reference that some execution could satisfy. We corroborate this empirically—zero `SC1101` on 193 unmodified workflows (Section 6.5). The analysis has two tiers that mirror the tool’s two layers. The **native** tier seeds the start document as $\top$ (the execution input is arbitrary), so findings arise only where the workflow itself constructs the data it later reads—a `Pass Result`, a `ResultSelector`, a static `Parameters` object, or an `OutputPath` that drops a field; the statement above holds *unconditionally* for this tier. The **typed** tier seeds the start as a closed record of the declared input schema (sidecar, DSL, or CNCF JSON Schema), which recovers the full “field never produced” class flow-sensitively through the `ResultPath` merge—strictly more than an adjacent-task field-subset check. There the guarantee is relative to a *closed-world reading* of the declared contract: the schema enumerates the fields the input may contain. This matches the intent of an author who declares a contract; where inputs are genuinely open, the native tier (which assumes nothing) is the sound fallback.

3.4 The annotation layer

Explicit semantics are supplied either through a *sidecar* file attached to an existing ASL workflow (Listing 1.2) or, for greenfield workflows, through a typed builder DSL that emits both the ASL and its annotations (one `.task(...)` call per state carries its schema, protocol edge, idempotency, persistence, and compensation).

The two routes resolve to the same internal representation, so the analyses are agnostic to how a workflow was authored.

Listing 1.2. A sidecar annotation (TOML), excerpt.

```
[schemas.OrderReserved]
fields = ["orderId", "amount", "reservationId"]
[tasks.ChargeCard]
idempotent = false
persistent = true
compensation = "RefundPayment"
state_in = "OrderReserved"
state_out = "OrderPaid"
```

3.5 Inference of default annotations

Most existing workflows carry no annotations, yet their task names and resource ARNs are highly conventional. STEP CHECK exploits this with a documented, configurable heuristic that classifies each task from a lowercased *signal* (its name, its **Resource**, and parameters such as **FunctionName**). The heuristic uses four small keyword sets: *read* verbs (**get**, **list**, **describe**, **query**, **validate**, ...) imply idempotent and non-persistent; *write* verbs (**create**, **charge**, **send**, ...) imply non-idempotent; *resource-acquiring* verbs (**reserve**, **pay**, **provision**, **putItem**, ...) additionally imply persistent; and *notification* verbs/resources (**notify**, **publish**, **:sns:**, ...) are non-idempotent but, being fire-and-forget, never persistent. When no keyword matches, the heuristic *abstains*, leaving the task unclassified rather than guessing. Inference thus yields a zero-effort default whose mistakes (Section 6) are visible (warnings) and correctable (an explicit annotation always wins).

3.6 Architecture

The pipeline is a straight line: a *frontend* lowers a surface syntax to a format-agnostic intermediate representation (IR); annotation resolution attaches declared and inferred semantics to the IR; a *pass* pipeline runs the eight analyses, each emitting coded diagnostics; and an *emitter* lowers verified workflows back to ASL. The design is modular along two axes: a new input format is a new **Frontend**, and a new check is a new **Pass** (Section 5)—indeed the three analyses new to this paper were each added as a single **Pass** with no change to the IR or the other checks.

4 Static Analyses

STEP CHECK runs eight independent passes over the IR, each emitting diagnostics with a stable code (Table 1), a severity, the offending state, and a remediation note. All are linear in the number of states (the data-flow pass, linear modulo the control-flow fixpoint) and recurse into nested **Map/Parallel** sub-machines.

Table 1. Diagnostic codes emitted by STEP CHECK. Severity $\dagger$ is *error* when derived from declared facts and *warning* when derived from inference.

Code	Pass	Meaning
SC0001–0010	structural	undefined start/transition target, unreachable state, dead-end, non-exhaustive Choice , empty Map/Parallel , malformed state
SC1001–1003	data-flow	waitForTaskToken without a task token, \$\$.Map misuse, invalid JSONPath payload
SC1010	data-flow	typed contract mismatch: successor requires a field the predecessor does not produce
SC1101	data-flow	reference to a field no execution can have produced (provenance; sound)
SC2001–2002	typestate	out-of-order composition; undeclared business-protocol transition
SC3001 $\dagger$	retry	non-idempotent task retried on a broad error class
SC4001–4002 $\dagger$	compensation	persistent task with no compensation/handling; compensation names a missing state
SC5001–5002	concurrency	Parallel branches / concurrent Map iterations write the same resource
SC6001–6003	temporal	unbounded callback; retry budget exceeds machine timeout; heartbeat $\geq$ timeout

Structural validation (SC0xxx). A reachability traversal from **StartAt** (following normal *and* error transitions) checks that all transition targets exist, that no state is unreachable, that every non-terminal offers a continuation, that a terminal is reachable, that **Choice** states declare a **Default**, and that **Map/Parallel** states are well-formed—sound checks that catch defects AWS reports only at run time.

Data-flow provenance (SC1xxx, SC1101). This is the analysis of Section 3.3, run as a forward abstract interpretation that computes each state’s document shape and checks the data references a state evaluates against it (its **Parameters/ItemSelector** ‘\$’ constructors and **ItemsPath**). A reference to a path that is *definitely absent* in the state’s shape is SC1101—a sound finding (no false positives). Three legacy ASL-binding checks are special cases the same pass still reports: a **waitForTaskToken** task must pass **\$\$.Task.Token** (SC1001); the **\$\$.Map** context is valid only inside a **Map** (SC1002); and every payload must be a valid path or **States.*** intrinsic (SC1003). SC1101 goes strictly beyond all of these and beyond what **statelint/asl-validator** check: it is the only one that asks whether the field a *valid* path names can ever have been produced. With a declared input schema the shape is closed and the analysis recovers the full **amount/total** class (Section 2.2); STEP CHECK additionally retains an adjacent-task field-subset check (SC1010) as a cheap complement when only per-task schemas, not a workflow input schema, are declared.

Typestate (SC2xxx). Using `state_in/state_out` labels, the pass checks that adjacent tasks compose (predecessor output state = successor input state, SC2001), catching re-orderings such as ship-before-pay, and that each task’s transition is a permitted protocol edge when one is declared (SC2002). Absent annotations it is a no-op.

Retry safety (SC3xxx). A task with a `Retry` policy whose `ErrorEquals` includes a broad class (`States.ALL/States.TaskFailed`) is flagged SC3001 if its idempotency resolves to *non-idempotent*. Retries restricted to transient errors (throttling, `ServiceException`) are not flagged, as they fire before any effect.

Compensation completeness (SC4xxx). A *persistent* task needs either a declared compensation or a `Catch` path; one with neither leaves its effect uncompensated (SC4001), and a compensation naming a non-existent state is SC4002.

Concurrency interference (SC5xxx). `Parallel` branches and `Map` iterations each receive their own copy of the state document, so the document cannot race—but the *external* resources their tasks touch can. The pass mines each task’s resource (its integration service plus the named target: `TableName`, `QueueUrl`, `TopicArn`, `Bucket`) and, to avoid false positives on idiomatic keyed/commuting patterns, considers only *overwriting* writes—`PutItem`/`PutObject`, which are last-writer-wins. It excludes `UpdateItem` (an atomic merge that commutes for disjoint attributes) and fire-and-forget `Publish`/`SendMessage` (independent messages). It then flags two patterns: two `Parallel` branches that both overwrite the *same* resource (SC5001); and a concurrent `Map` (`MaxConcurrency` $\neq 1$) whose iterations overwrite a shared resource while threading *no* per-item data (no ‘\$’ reference in the write, so every iteration clobbers the same item, SC5002). Both are *warnings*, consistent with the tool’s confidence model. No prior ASL tool reasons about cross-branch interference.

Retry / timeout budget (SC6xxx). ASL’s resilience knobs interact in ways the platform never cross-checks. A `waitForTaskToken` (or `Activity`) task with neither `TimeoutSeconds` nor `HeartbeatSeconds` waits up to a year for a token that may never arrive (SC6001, the most common in-the-wild finding—human-approval and external-callback steps). When the state machine declares a `TimeoutSeconds`, the pass computes a task’s worst-case retry budget as the geometric back-off sum $\sum_k \text{interval} \cdot \text{backoff}^k$ and flags retries that cannot complete within it (SC6002). Finally `HeartbeatSeconds` $\geq$ `TimeoutSeconds` is a definite mis-configuration the platform rejects only at deploy time (SC6003).

Worked example. `STEPCHECK` reports no findings on the well-formed order workflow. Reordering it to ship before charging (Section 2.2) yields five errors and no false positives: two SC2001 typestate mismatches (e.g. `ShipOrder` consumes `OrderPaid` but its new predecessor produced `OrderReserved`) and three SC1010 contract mismatches (`paymentId`, `amount`, `reservationId` required but not produced upstream)—complementary checks: typestate explains the ordering fault,

the contract check pinpoints the broken data. The native data-flow tier needs no annotations at all: on a workflow that constructs `{order:{orderId,amount}}` in a `Pass` and then reads `$.order.total`, `STEPCHECK` reports SC1101 directly—the `amount/total` bug caught on raw ASL, which `statelint` accepts as a valid path.

5 Implementation

`STEPCHECK` is implemented in Rust as a single command-line tool, `stepcheck` (3,692 lines). We chose Rust deliberately: a standalone analyzer with a `serde`-based parser is what makes it possible to ingest the *existing* ASL JSON found in real repositories—rather than only workflows authored in a new embedded language—and it yields a single dependency-light binary with predictable, sub-millisecond performance.

5.1 A modular frontend/pass pipeline

Extensibility rests on two traits (Listing 1.3): a `Frontend` lowers a surface syntax to the IR, a `Pass` reads only the IR and writes diagnostics. Adding an input format is a new `Frontend`; a check, a new `Pass`. We exercised this directly—the tool ships three frontends (typed DSL, raw ASL, CNCF [7]), the last added *without changing the IR or any pass* (Section 6.9).

Listing 1.3. The two extension points.

```
pub trait Frontend { fn load(&self, src: &str) -> Result<Workflow>; }

pub trait Pass {
    fn id(&self) -> &'static str;
    fn run(&self, wf: &Workflow, sink: &mut DiagnosticSink);
}
```

5.2 Components

The **workflow IR** is a format-agnostic graph of states (`Task`, `Choice`, `Parallel`, `Map`, `Wait`, `Pass`, `Succeed`, `Fail`) with control flow, JSONPath data-flow fields, retry/catch policies, nested sub-machines, and a slot for resolved annotations. The **Asl frontend** lowers from a generic `serde_json` value rather than rigid structs—so the diversity of real ASL (intrinsic, `.$` parameters, the `$$` context, SAM `${}` placeholders, classic `Iterator` vs. distributed `ItemProcessor`, JSONata) never causes a hard failure—and parses all 193 corpus workflows without error. **Annotation resolution** loads a TOML sidecar, resolves schema names to field sets, and fills gaps with the heuristic of Section 3.5. The **passes and diagnostics** implement the eight analyses plus an engine that renders findings for a terminal or as JSON; the data-flow pass is the largest at ≈ 400 lines (the shape lattice, transfer functions, and control-flow fixpoint), the rest a few dozen lines each. The **Asl emitter** lowers a verified IR back to deployable ASL JSON; it is the

DSL frontend’s code-generator and the basis of the AWS round-trip (Section 6)—deploying its output surfaced (and we fixed) a real bug where an explicit `End` was emitted on `Succeed/Fail` states, which Step Functions rejects.

CLI and error injection. `stepcheck` exposes `check`, `infer`, `scan`, `stats`, `eval`, `mutate`, and `demo` (the last emits the built-in typed-DSL example to ASL). The `mutate` command injects exactly one defect of a chosen class into a workflow—a broken JSONPath binding, a reference redirected to an absent field (data-flow), a broad retry on a non-idempotent task, a removed compensation, a dangling transition, two `Parallel` branches made to write one resource, or a heartbeat $\geq$ timeout—and re-emits deployable ASL; it drives the mutation study of Section 6.

The **3,692** lines divide as: workflow IR 298; three frontends (ASL parse/emit, typed DSL, CNCF) 729; annotations + inference 258; the eight analysis passes + manager 1,426; diagnostics 115; mutation engine 309; and CLI + statistics + evaluation harness 557.

6 Evaluation

We evaluate STEP CHECK on real serverless workflows, asking: can it detect the four defect classes before deployment (RQ1); can it recover the semantics ASL omits cheaply enough to analyse existing workflows (RQ2); and what does verification cost, at build and run time (RQ3)? Every number below is produced by the released tool over a committed corpus and is reproducible with the commands shown.

6.1 Setup and corpus (RQ1)

We mined **193** Step Functions workflows from two public AWS repositories (`aws-samples/aws-stepfunctions-examples` and `step-functions-workflows-collection`), taking every file that parses as an ASL state machine. The corpus (Fig. 1) is realistic and diverse: 1,355 states, median 6 per workflow (max 33), and broad use of `Choice` (166), `Map` (64), `Parallel` (33), retries (84 workflows), and catch handlers (51). All experiments ran on a commodity laptop (Rust 1.96 release build).

6.2 Findings in the wild (RQ1)

Run unmodified over the corpus (with inference enabled), STEP CHECK flags **95 of 193** workflows. We separate *sound native* findings, which need no annotations, from *inference-driven* findings, which are warnings requiring triage. Among the native findings, nine workflows contain a `Choice` with no `Default` (SC0007)—a recognised cause of `States.NoChoiceMatched` run-time failures—and one is genuinely malformed (SC0010): an `aws-samples` example nests a top-level `QueryLanguage` directive *inside* its `States` block, where Step Functions would reject it. The

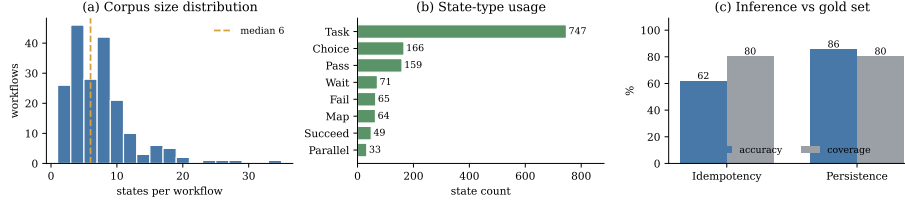


Fig. 1. Evaluation overview. (a) Distribution of workflow sizes and (b) state-type usage across the 193-workflow corpus; (c) inference accuracy and coverage for idempotency and persistence against the agreement-checked gold set (Section 6.6).

temporal analysis adds **33** findings, all **SC6001**: unbounded `waitForTaskToken` callbacks—human-approval and payment/shipping steps with no timeout or heartbeat, a real anti-pattern AWS documents against. The data-flow (**SC1101**) and concurrency (**SC5xxx**) analyses fire on *none* of the 193: **SC1101** because the analysis is sound and these samples are correct (Section 6.5), and **SC5xxx** because its conservative overwriting-write rule finds no genuine last-writer-wins race among the corpus’s (correctly keyed or commuting) concurrent writes—both precision results, with detection power shown by the mutation study below. The inference-driven warnings (**SC3001**: 36; **SC4001**: 117) mark non-idempotent tasks retried broadly and persistent tasks lacking a compensation path; as Section 6.6 shows, these have measured precision of 72% on gold-covered findings (81% for the compensation warnings) but include false positives (e.g. a `RefundPayment` compensator inferred as needing its own compensation), which is precisely why they are surfaced as warnings rather than errors.

6.3 Mutation-based detection (RQ1)

Because correct real workflows expose few of the deeper defects, we measure detection power by *error injection*. For each defect class we inject exactly one defect per applicable workflow with `stepcheck mutate` and re-verify; a defect counts as *detected* only if the expected diagnostic’s count strictly increases over the unmutated baseline (so we measure detection of the *injected* defect, not pre-existing findings). Across **616** mutants spanning six classes—the four above plus a shared-resource **Parallel** write (**SC5001**) and a heartbeat $\geq$ timeout (**SC6003**)—**STEPCHECK** achieves **100% recall** (Table 2, **STEPCHECK** column), and it never reports the expected defect on the corresponding originals. (The data-flow class is studied separately in Section 6.5 because, unlike the others, its detection depends on a known document shape rather than firing on raw ASL.) We stress that this measures *reliability of each check on the defect class it targets*, not detection of arbitrary bugs; the mutations are constructed to fall within each check’s remit. The next subsection contrasts this with what a standard validator detects on the same mutants.

Table 2. Injected-defect detection over 616 mutants (delta over baseline): STEPCheck vs. `statelint`. A mutant counts as detected only when the tool emits a new problem naming the injected fault.

Defect class	Check	Applicable	STEPCheck	<code>statelint</code>
Dangling transition	SC0002	166	166 (100%)	166 (100%)
Broken data binding	SC1003	141	141 (100%)	140 (99%)
Unsafe retry	SC3001	103	103 (100%)	0 (0%)
Missing compensation	SC4001	19	19 (100%)	0 (0%)
Concurrency interference	SC5001	21	21 (100%)	0 (0%)
Temporal (heartbeat/timeout)	SC6003	166	166 (100%)	0 (0%)
Total		616	616 (100%)	306 (50%)

6.4 Comparison to a standard validator (RQ1)

The natural baseline is the validation a Step Functions developer already has. `statelint` [3] (the AWS Labs reference linter; we ran v0.8.0) checks an ASL definition against the J2119/ASL schema plus structural rules. We ran it on the same corpus and the same 616 mutants, scoring a mutant *detected* only when `statelint` emits a new problem that names the injected fault (Table 2). The two tools *agree* on the defects `statelint` can express: it flags every dangling transition (166/166) and—because a broken JSONPath binding violates the schema—almost every broken *binding* (140/141). But it detects **0** unsafe retries, **0** missing compensations, **0** concurrency-interference defects, and **0** temporal-budget defects: a broad `States.ALL` retry, an absent `Catch`, two branches writing one table, and a heartbeat that exceeds a timeout are all schema-valid, so a validator without a notion of idempotency, persistence, resources, or a temporal budget cannot see them. (A naive count-of-problems metric would appear to “detect” many of these—the injected retry’s float `IntervalSeconds` once tripped `statelint`’s integer pedantry, a removed `Catch` orphans its handler state—but none of those complaints is the analysis in question, so we do not credit them.) In the wild the asymmetry is starker still: `statelint` flags 113/193 workflows with 499 problems, **410 of them** schema-shape nits (e.g. `BackoffRate/IntervalSeconds` number typing) and none touching the six semantic classes, whereas STEPCheck’s findings are all semantically meaningful. STEPCheck is thus complementary to and strictly more discriminating than schema validation on the properties that motivate it.

6.5 Data-flow provenance: soundness, power, and reach (RQ1/RQ2)

The data-flow analysis (Section 3.3) is the paper’s central new check, and its claim is *soundness*—no false positives—so we evaluate it on its own terms. *Precision*. Run over all 193 unmodified workflows, SC1101 fires **zero** times. These are mostly correct sample workflows, and the analysis is conservative by construction; firing on none of them is the expected, desired outcome and empirically corroborates

the soundness statement of Section 3.3. *Power*. That the check is not vacuous we show on worked demonstrators (released in the corpus). On a *native* example—no annotations—where a `Pass` builds `{order:{orderId,amount}}` and a later task reads `$.order.total`, `STEPCHECK` reports `SC1101` on raw ASL; `statelint` and `asl-validator` accept it (the path is valid). On a *typed* example with a three-field input schema, it reports two `SC1101`: a schema miss (`$.total` for `amount`) and a *flow-sensitive* miss—`$.reservation.reservationCode` where an upstream `ResultSelector/ResultPath` merge produced `reservationId`—a defect an adjacent-task field-subset check cannot see because the field is created two states earlier. *Reach*. To measure detection at corpus scale we run the typed tier: each workflow’s start document is seeded as a closed record of the top-level fields it references, and we inject a missing-field defect per workflow (redirecting a reference to an absent field). `STEPCHECK` detects **84 of 126 (67%)**; the undetected third are references that sit *downstream of an opaque task result*, where the document shape has soundly collapsed to $\top$ and no missing-field claim is possible. This number is therefore not a reliability gap but a measurement of ASL’s intrinsic opacity: roughly two-thirds of field references on real workflows are statically recoverable, and the remainder are exactly what richer typed outputs (a `ResultSelector`, or CNCF’s declared output schemas, Section 6.9) push back. The baseline produces no spurious `SC1101` under the closed-world seed either, so the typed tier inherits the native tier’s precision.

6.6 Inference accuracy (RQ2)

The semantic checks depend on recovering idempotency and persistence. We hand-labelled a gold standard of **147 tasks across 23 workflows**—spanning Sagas, an inventory domain, account vending, security automation, media/AI pipelines, CQRS, data migration, and more—by each task’s true operational semantics. To guard against labeller bias we labelled the new workflows in *two independent passes* under a written rubric and reconciled the differences; agreement was high (Cohen’s $\kappa = 1.00$ for idempotency and 0.75 for persistence over the doubly-labelled tasks), and we release the labels for scrutiny. The heuristic abstains on $\sim 20\%$ of tasks (unconventional verbs such as *confirm*, *audit*, *aggregate*); on the rest it reaches **85.6%** accuracy for persistence but only **61.9%** for idempotency (Table 3). The asymmetry is the headline result of this experiment, and it is honest: persistence (which drives the compensation check) tracks resource-acquiring verbs well, whereas idempotency is genuinely hard from names alone—*cancel/release* compensators and stable-key writes (a DynamoDB `PutItem/UpdateItem` keyed by a primary key) are idempotent *in effect* but read as writes. That high inter-annotator agreement coexists with low heuristic accuracy means the truth is clear but the names do not carry it. This is precisely why inferred findings are *warnings*: a one-line annotation overrides any default, and the residual error is visible, not silent.

Precision of in-the-wild warnings. The gold set lets us measure the precision of the inference-driven warnings of Section 6.2 directly: for the 39 `SC3001/SC4001`

Table 3. Inference accuracy vs. the agreement-checked gold set (147 tasks, 23 workflows; accuracy is over non-abstained predictions).

Property	Labelled	Predicted	Abstained	Correct	Accuracy	Coverage
Idempotency	147	118	29	73	61.9%	80.3%
Persistence	147	118	29	101	85.6%	80.3%

findings that land on a gold-labelled task, a finding is a true positive iff the gold label confirms it. **28/39 (72%)** are true positives—**26/32 (81%)** for the compensation warnings SC4001, but only **2/7** for the broad-retry warnings SC3001, mirroring the weaker idempotency inference that drives them. Every false positive is a compensator (`RefundPayment`, `CancelFlightReservation`) or an idempotent keyed/metadata write—the same patterns above—which is exactly why these are surfaced for triage rather than as errors. RQ2 answer: persistence is recovered well and compensation warnings are mostly actionable; idempotency-driven retry warnings are noisier and benefit most from a declared annotation.

6.7 Verification cost and runtime overhead (RQ3)

Verification is cheap: the eight passes—including the data-flow fixpoint—average $\approx 39 \mu\text{s}$ /workflow (median $\approx 25 \mu\text{s}$, max $\approx 1.1 \text{ ms}$); the whole corpus verifies in $\approx 7.5 \text{ ms}$. The three new analyses roughly doubled the per-workflow cost over the five-pass version yet kept it firmly sub-millisecond on all but one workflow. Because all analysis is ahead-of-deployment and the output is ordinary ASL, the *runtime* overhead is **zero**—no middleware, sidecar, or instrumentation—unlike runtime fault-tolerance and transaction approaches [21,6].

6.8 Conciseness and AWS round-trip

The typed DSL is also concise: 20 declarative statements expand to a 119-line ASL definition while additionally encoding five schemas, a four-edge protocol, and per-task idempotency/persistence/compensation that ASL cannot express. To confirm compatibility end-to-end, we deployed the DSL-verified order workflow to real AWS Step Functions *unmodified* (only binding `FunctionName` to a Lambda ARN) as an Express machine: it reached SUCCEEDED, the input propagating through all four steps, billed 500 ms / 64 MB; the resources were then torn down. A verified STEPCHECK workflow runs on the unmodified industrial platform.

6.9 Cross-format generalization: CNCF Serverless Workflow

To test generalization beyond AWS, we added a third frontend for the vendor-neutral CNCF Serverless Workflow DSL [7] and ran it on **66** example workflows from the specification’s repository. The frontend lowers the DSL’s tasks

(`call/do/fork/switch/try/...`) to the *same* IR *without changing the IR or any analysis pass*, and parses all 66 with no failures. The structural, typestate, retry-safety, compensation, concurrency, and temporal analyses apply unchanged; only the ASL-specific JSONPath references the data-flow pass mines stay silent—correctly, since CNCF uses runtime expressions, not JSONPath, so SC1101 and the binding checks (SC1001–SC1003) do not fire. A mutation study detects every injected defect with an injection site (retry 4/4, structural 9/9, concurrency 1/1, temporal 50/50). Crucially, CNCF can declare JSON Schemas, so where it does the typed contract check (SC1010) runs at *full precision with no inference*: a reordered schema-annotated order workflow is rejected with three SC1010 errors from the declared schemas—the same defect inference only approximates on raw ASL, here exact. The asymmetry is informative: CNCF makes data contracts explicit but, like ASL, leaves idempotency and compensation implicit, so those checks still rely on annotation or inference in both ecosystems.

6.10 Threats to validity

Construct. The mutation study reports 100% recall by design: each mutation targets a defect within a check’s remit, so the result attests to the reliability of each check, not to coverage of all possible bugs. We report it alongside the in-the-wild findings, which are not constructed. *Internal.* Gold labels for idempotency/persistence are judgements; we mitigated bias by labelling in two independent passes under a written rubric with high agreement ($\kappa = 1.00/0.75$), documented the criteria, and release the labels for scrutiny. The contested cases (cancel, refund, keyed writes) are exactly where the paper claims inference is imperfect, and we report the resulting warning precision rather than hide it. *External.* The corpus is drawn from AWS sample repositories, which may over-represent didactic workflows; we mitigate this by taking *all* parseable workflows rather than a curated subset, yielding a realistic spread of sizes and features. *Soundness.* The data-flow and structural checks are conservative (they flag only definite faults), corroborated by zero SC1101 false positives on the corpus; typestate and typed-contract precision depend on annotation quality, which is why STEP CHECK reports inferred findings as warnings. The concurrency and temporal-budget analyses are *warnings*: the concurrency check is deliberately conservative (only last-writer-wins overwrites, never commuting or item-keyed writes), which is why it raises zero in-the-wild false positives while still detecting every injected race; a missing callback timeout (SC6001) may occasionally be acceptable for a short-lived task, so it too prompts review rather than asserting a bug. The budget check SC6002 did not fire in the wild (no corpus workflow declares a machine timeout tight enough to be overrun); we keep it as a defensive check and leave a labelled-precision study of the warning classes to future work.

7 Related Work

Serverless orchestration, durable execution, and transactions. The commercial orchestrators and their trade-offs have been analysed extensively [20,13], and

many programming models make orchestration more expressive or data-aware [6]. Durable-execution engines (Temporal, Cadence [18,19]) and systems like Beldi [21] provide exactly-once execution and transactional/Saga semantics *at run time*, via logs and recovery protocols. All target execution—performance, scalability, convenience, fault tolerance. STEP CHECK is complementary: it targets *correctness*, statically, on the most widely deployed platform, adding no runtime machinery; a verified workflow can still run on a durable backend.

Typestate, sessions, and Saga compensation. Typestate originates with Strom and Yemini [17] and extends to objects and API protocols [10]; session and multiparty session types give a rich theory of protocol conformance [12]. STEP CHECK adapts the core idea—valid-state-only composition—to workflow tasks, trading full generality for a decidable, linear-time check inside an industrial toolchain. The Saga pattern [11] underpins cloud-native recovery [14], but compensation is usually coded and validated at run time [8]; STEP CHECK adds a *static* completeness check—every persistent operation must have a reachable compensator before shipping.

Data-flow analysis and ASL tooling. The validators available for ASL—`statelint` [3], `asl-validator`, and AWS’s `ValidateStateMachineDefinition`—check the J2119/ASL JSON schema, transition existence, and JSONPath *syntax*; none reasons about field provenance. Our data-flow pass casts ASL’s I/O-processing pipeline as an abstract interpretation [9] over a may-present lattice of document shapes, yielding a *sound* (no-false-positive) provenance check—to our knowledge the first static data-flow analysis for the language. It is the workflow-internal analogue of the producer/consumer reasoning that consumer-driven contracts and static REST-client checks apply *across* services [15].

Concurrency and workflow verification. Interference and coordination for concurrent objects and distributed transactions are studied for actors and state-machine DSLs—interference-free network objects, and path-sensitive atomic commit that reasons about whether two messages’ effects commute [16]—but not for serverless-workflow `Parallel/Map` fan-out, which our concurrency pass targets. Workflow soundness is verified with Petri nets [1,5] and service compositions with model checking [4]: strong, general guarantees that require dedicated formalisms and are rarely wired into everyday cloud development. STEP CHECK deliberately targets a deployed industrial language with lightweight, linear checks and direct compilation. Against representative approaches—AWS Step Functions [2] and CNCF [7] (no static semantic checks), Temporal/Cadence [18,19] and Beldi [21] (runtime retry/Saga, not static), Petri-net/BPEL verification [5,4] (static but heavyweight, non-AWS), and session/typestate types [12,17] (typestate only)—STEP CHECK is, to our knowledge, the first to combine sound data-flow provenance, typestate conformance, retry safety, concurrency-interference and temporal-budget checking, and compensation completeness in a single *static* verifier that compiles directly to executable AWS Step Functions.

8 Conclusion

Serverless workflow orchestration has become a backbone of cloud-native systems, yet its workflows are untyped configuration whose correctness is checked only when they run—and the tooling developers have stops at JSONPath *syntax*. We presented STEP CHECK, a static verifier whose centrepiece is the first *sound* data-flow analysis of the Amazon States Language: a flow-sensitive abstract interpretation over a may-present lattice of document shapes that catches missing-field references with no false positives and no annotations. Around it STEP CHECK adds seven further checks—structural, typed contracts, typestate, retry safety, concurrency interference, retry/timeout budget, and Saga compensation—two of them for classes no ASL tool had addressed, all compiling back to executable ASL. The key to analysing the workflows that already exist is a two-layer model: sound native checks that need nothing beyond ASL, paired with a lightweight annotation layer whose defaults are inferred from the naming and resource conventions that pervade real workflows.

Our evaluation on 193 real Step Functions workflows (plus 66 CNCF) supports the three research questions. STEP CHECK detects the targeted defect classes (100% recall over 616 injected defects) where a standard schema validator detects none of the retry, compensation, concurrency, or temporal defects, and surfaces 33 unbounded-callback defects in unmodified workflows; the data-flow and concurrency analyses fire zero false positives in the wild (RQ1). It recovers persistence at 86% and idempotency at 62% accuracy at zero authoring cost against an agreement-checked gold set—enough to make compensation warnings mostly actionable (81% precision)—all surfaced as correctable warnings, and its typed tier recovers two-thirds of injected missing-field defects, the remainder being ASL’s intrinsic opacity (RQ2); and it verifies a workflow in $\approx 39 \mu\text{s}$ while adding zero runtime overhead, with verified workflows running unmodified on real AWS Step Functions (RQ3).

The broader message mirrors the history of type systems: just as static typing moved classes of error from run time to compile time, lightweight static verification—rooted in a sound data-flow core—can do the same for serverless orchestration, on the platforms developers already use. Future work includes further frontends (SAM, CDK), richer effect/ownership information to sharpen the compensation and concurrency analyses, a precise element-level shape for `Map/Parallel` item contexts, and a labelled-precision study of the concurrency and temporal warnings. We release STEP CHECK, the corpus, and the evaluation harness to support replication and reuse.

References

1. van der Aalst, W.M.P.: The application of Petri nets to workflow management. *Journal of Circuits, Systems and Computers* **8**(1), 21–66 (1998). <https://doi.org/10.1142/S0218126698000043>

2. Amazon Web Services: AWS step functions developer guide (amazon states language). <https://docs.aws.amazon.com/step-functions/> (2026), accessed 2026-06-17
3. Amazon Web Services Labs: statelint: A validator for Amazon states language documents. <https://github.com/aws-labs/statelint> (2024), apache-2.0; v0.8.0. Accessed 2026-06-18
4. Bianculli, D., Ghezzi, C., Spoletini, P.: A model checking approach to verify BPEL4WS workflows. In: IEEE Int. Conf. on Service-Oriented Computing and Applications (SOCA). pp. 13–20 (2007). <https://doi.org/10.1109/SOCA.2007.5>
5. Blondin, M., Mazowiecki, F., Offtermatt, P.: Verifying generalised and structural soundness of workflow nets via relaxations. In: Int. Conf. on Computer Aided Verification (CAV). pp. 435–458 (2022). https://doi.org/10.1007/978-3-031-13188-2_23
6. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C.S.: Durable functions: Semantics for stateful serverless. Proc. ACM Program. Lang. **5**(OOPSLA) (2021). <https://doi.org/10.1145/3485510>
7. Cloud Native Computing Foundation: Serverless workflow specification. <https://serverlessworkflow.io/> (2026), accessed 2026-06-17
8. Colombo, C., Pace, G.J.: Comprehensive monitor-oriented compensation programming. arXiv preprint (2014), arXiv:1404.0849
9. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL). pp. 238–252. ACM (1977)
10. DeLine, R., Fähndrich, M.: Typestates for objects. In: European Conf. on Object-Oriented Programming (ECOOP). pp. 465–490 (2004). https://doi.org/10.1007/978-3-540-24851-4_21
11. Garcia-Molina, H., Salem, K.: Sagas. In: ACM SIGMOD Int. Conf. on Management of Data. pp. 249–259 (1987). <https://doi.org/10.1145/38713.38742>
12. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL). pp. 273–284 (2008). <https://doi.org/10.1145/1328438.1328472>
13. Kulkarni, V., Reddy, N., Khare, T., Prasad, A.S., Babu, C., Simmhan, Y.: Characterizing FaaS workflows on public clouds: The good, the bad and the ugly. arXiv preprint (2025), arXiv:2509.23013
14. Mohammad, M.: Resilient microservices: A systematic review of recovery patterns, strategies, and evaluation frameworks. arXiv preprint (2025), arXiv:2512.16959
15. Seco, J.C., Ferreira, P., Lourenço, H., Ferreira, C., Ferrão, L.: Robust contract evolution in a typesafe microservices architecture. The Art, Science, and Engineering of Programming **4**(3) (2020). <https://doi.org/10.22152/programming-journal.org/2020/4/10>
16. Soethout, T., van der Storm, T., Vinju, J.J.: Path-sensitive atomic commit: Local coordination avoidance for distributed transactions. The Art, Science, and Engineering of Programming **4**(1) (2020)
17. Strom, R.E., Yemini, S.: Typestate: A programming language concept for enhancing software reliability. IEEE Transactions on Software Engineering **SE-12**(1), 157–171 (1986). <https://doi.org/10.1109/TSE.1986.6312929>
18. Temporal Technologies: Temporal: Durable execution and workflow orchestration. <https://temporal.io/> (2026), accessed 2026-06-17
19. Uber Technologies: Cadence: Fault-oblivious stateful workflow orchestration. <https://cadenceworkflow.io/> (2026), accessed 2026-06-17

20. Wen, J., Liu, Y.: An empirical study on serverless workflow service. arXiv preprint (2021), arXiv:2101.03513
21. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: USENIX Symp. on Operating Systems Design and Implementation (OSDI) (2020), arXiv:2010.06706